Na poprzednich zajęciach:

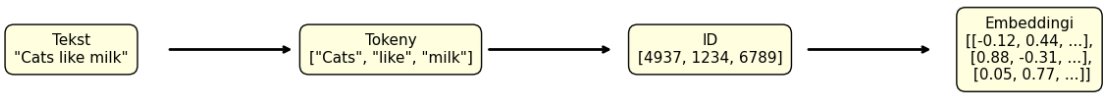
- **tokenizacja** – podział tekstu na słowa / tokeny,
- **spaCy Matcher** – znajdowanie wzorców na podstawie części mowy i cech morfologicznych.

Wektory słów

Tokenizacja a wektory słów

Tokenizacja -> Podziel tekst na kawałki (ID ze słownika)

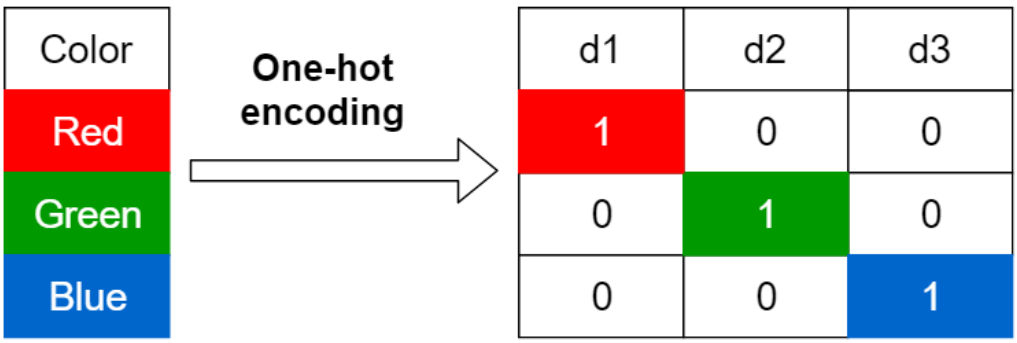
Embedding -> Przedstaw każde słowo w postaci liczb, aby algorytm mógł z nim dalej pracować



Komputer nie „rozumie" tekstu, tylko operuje na liczbach. Zatem każde słowo (np. „król", „kot", „miłość") trzeba jakoś zamienić na liczby, żeby można było nim manipulować w modelu.

Najprostszy pomysł — **one-hot encoding**

```
słownik = [kot, król, królowa]
"król" → [0, 1, 0]
```



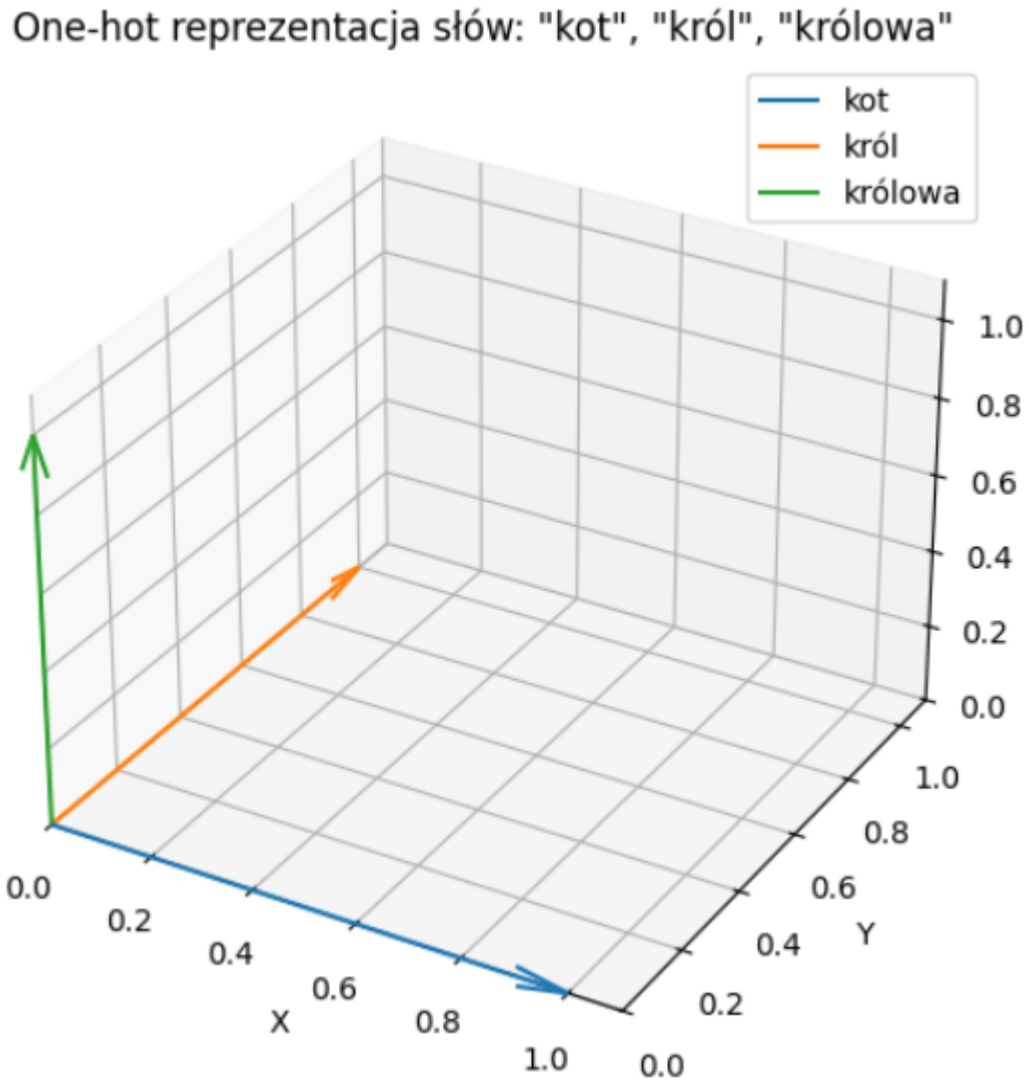
Dlaczego one-hot encoding jest problematyczny?

❌ Bardzo duży wymiar - Jeśli mamy słownik np. 100 000 słów, każdy wektor ma długość 100 000 — to ogromne marnotrawstwo pamięci.

❌ Brak informacji o znaczeniu - Wszystkie wektory są od siebie tak samo odległe — np. “kot” i “pies” są tak samo różne jak “kot” i “samolot”. Nie ma żadnego pojęcia o podobieństwie semantycznym (znaczeniowym).

❌ Brak możliwości uogólniania - Model nie może „zrozumieć”, że pewne słowa mają podobne znaczenie — każde słowo jest całkowicie niezależne.

```
słownik = [kot, król, królowa]
"kot" → [1, 0, 0]
"król" → [0, 1, 0]
"królowa" → [0, 0, 1]
```



Widzimy, że każdy wektor idzie w swoją stronę, każdy jest tak samo odległy od siebie

(90 stopni). Żadne dwa nie są do siebie bliżej ani dalej — odległość między każdym z nich jest taka sama.

W naszym przykładzie: kot, król i królowa są tak samo podobni i różni = nie ma wartości semantycznej.

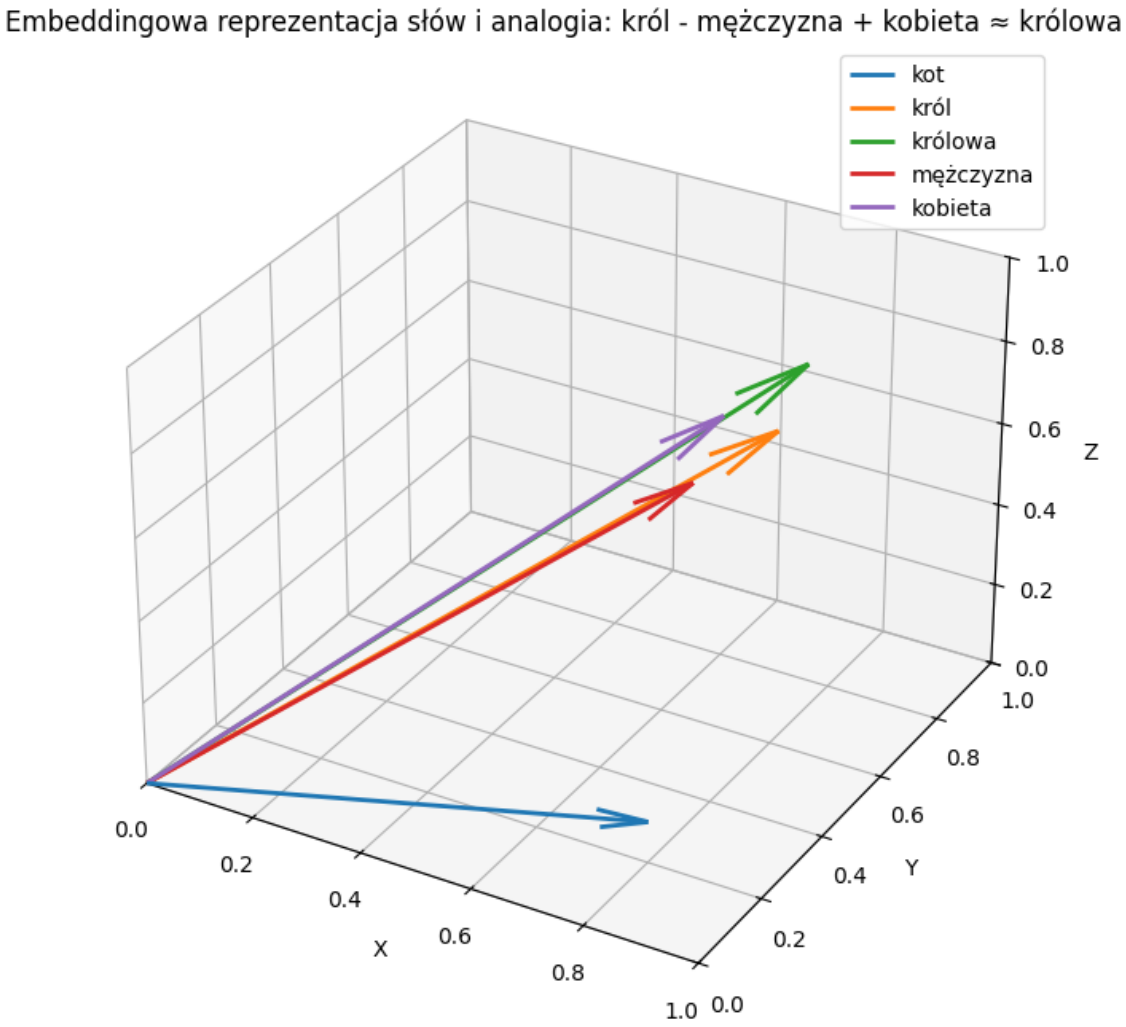
Embedingi

Embeddingi rozwiązują ten problem. W embeddingach uczymy model, żeby słowa, które występują w podobnych kontekstach, miały **podobne wektory**.

Dwa wektory są podobne, kiedy wskazują w podobnym kierunku w przestrzeni wielowymiarowej. Czyli: jeśli narysujesz te wektory jako strzałki zaczynające się w tym samym punkcie (np. w punkcie (0,0,0)) to mają **mały kąt między sobą**.

Przykład:

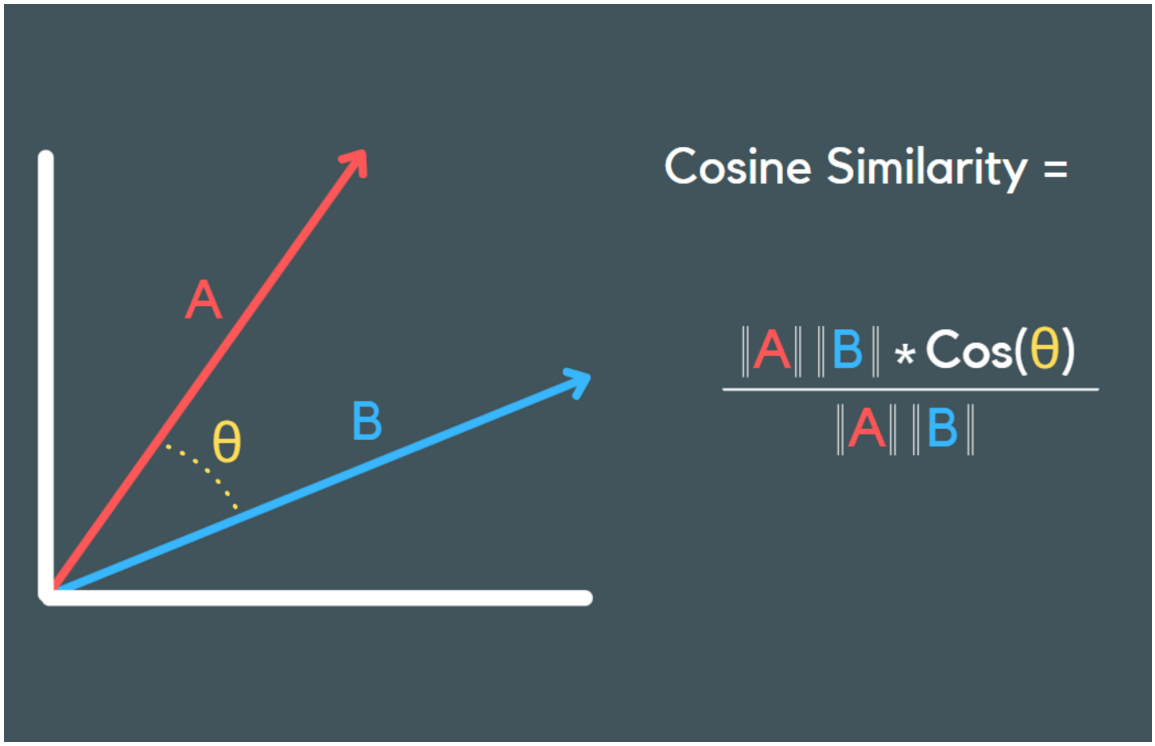
```
"king" → [0.25, 0.88, −0.12, 0.40, ...]
"queen" → [0.22, 0.90, −0.10, 0.45, ...]
"cat" → [−0.70, 0.11, 0.45, 0.20, ...]
```




```
król − mężczyzna + kobieta ≈ królowa
```

Teraz punkty w przestrzeni układają się tak, że słowa o podobnym znaczeniu są blisko siebie. To jest właśnie **semantyczne podobieństwo** — podobieństwo w sensie znaczenia, a nie liter czy indeksów w tabeli.

Cosinusowe podobieństwo — jak mierzymy „bliskość znaczeń”



Cosinusowe podobieństwo to sposób na sprawdzenie, czy dwa wektory są „w tym samym kierunku":

- 1 → bardzo podobne (np. „kot" i „pies")
- 0 → brak podobieństwa (np. „kot" i „okno")
- -1 → całkowite przeciwieństwo

Ważne: Patrzymy nie na długość wektorów, tylko na kąt między nimi. Dwa wektory, które „patrzą" w tym samym kierunku, oznaczają słowa o podobnym znaczeniu.

Skąd się biorą te wektory?

Z treningu modelu językowego (np. Word2Vec, GloVe, FastText, BERT). Model uczy się na dużych korpusach tekstu, obserwując konteksty słów.

Efekt:

„king" i „queen" często występują w podobnych zdaniach (np. royal palace, throne, reign) → mają podobne wektory.

„dog" i „cat" też → inna grupa semantyczna.

Trochę praktyki

Zaczniemy od zapoznania się z biblioteką **gensim** Jej nazwa pochodzi od "GENerate SIMilar" — bo jej głównym celem jest znajdowanie podobieństwa między dokumentami lub słowami.

Użyjemy klasy **KeyedVectors** - to strukturą danych w gensim, która przechowuje wektory słów (embeddingi) wytrenowane dowolnym modelem. Czyli po prostu taki kontener do pracy z wektorami - niezależnie od tego, jak one zostały wytrenowane.

In [1]:
```
!pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp313-cp313-macosx_11_0_arm64.whl.metadata (8.4 k
B)
Collecting numpy>=1.18.5 (from gensim)
  Downloading numpy-2.5.4-cp313-cp313-macosx_14_0_arm64.whl.metadata (6.6 k
B)
Collecting scipy>=1.7.0 (from gensim)
  Downloading scipy-1.18.1-cp313-cp313-macosx_14_0_arm64.whl.metadata (62 k
B)
Collecting smart_open>=1.8.1 (from gensim)
  Downloading smart_open-8.0.3-py3-none-any.whl.metadata (25 kB)
Collecting wrapt (from smart_open>=1.8.1->gensim)
  Downloading wrapt-2.5.1-cp313-cp313-macosx_11_0_arm64.whl.metadata (7.6 k
B)
Downloading gensim-4.4.0-cp313-cp313-macosx_11_0_arm64.whl (24.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.4/24.4 MB 15.4 MB/s  0:00:01
15.5 MB/s eta 0:00:01
Downloading numpy-2.5.4-cp313-cp313-macosx_14_0_arm64.whl (5.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.4/5.4 MB 11.3 MB/s  0:00:00 12
.5 MB/s eta 0:00:01
Downloading scipy-1.18.1-cp313-cp313-macosx_14_0_arm64.whl (20.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 20.5/20.5 MB 16.0 MB/s  0:00:01a
0:00:01[36m0:00:0101
Downloading smart_open-8.0.3-py3-none-any.whl (73 kB)
Downloading wrapt-2.5.1-cp313-cp313-macosx_11_0_arm64.whl (106 kB)
Installing collected packages: wrapt, numpy, smart_open, scipy, gensim
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5/5 [gensim]━━━━ 4/5 [gensim]
Successfully installed gensim-4.4.0 numpy-2.5.4 scipy-1.18.1 smart_ope
n-8.0.3 wrapt-2.5.1

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
```

In [2]:
```
import gensim
from gensim.models import KeyedVectors
```

weźmiemy sobie polskie pre-trenowane wektory: http://dsmodels.nlp.ipipan.waw.pl
pobieramy plik:

In [5]:
```
!curl -k -L -O http://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-re
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curr
ent
                                 Dload  Upload   Total   Spent    Left  Spee
d
100   389  100   389    0     0   6392      0 --:--:-- --:--:-- --:--:--  64
83
100 2052M  100 2052M    0     0  12.4M      0  0:02:44  0:02:44 --:--:-- 1
3.2M
```

Rozpakowujemy dane

In [6]:
```
!gzip -d "nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz"
```

Badamy format pliku

In [73]:
```
#Zakomentowałem, żeby nie było kilkunastu kartek tylko z tym

#!head -n 100 nkjp+wiki-forms-restricted-300-skipg-ns.txt
#Pierwsza linia - lość słów (1982479), rozmiar wektora(300)
#Każda kolejna linia - słowo, liczby reperezentujące wektor
```

(to może chwilę zająć)

In [3]:
```
#Ładowanie wektorów z pliku do naszego KeyedVectors
# Zmieniłem ze względu na brak wystarczajcacej pamieci
wv = KeyedVectors.load_word2vec_format('nkjp+wiki-forms-restricted-300-skipg
wv.save('nkjp.kv')
# wv = KeyedVectors.load('nkjp.kv')
```

In [5]:
```
wv.key_to_index.keys()
wv['dom']
```

Out[5]:
```
array([-0.39294 , -0.156333,  0.017816, -0.208752,  0.442839, -0.389071,
       -0.22354 ,  0.119761,  0.201861,  0.136966,  0.005829, -0.036992,
       -0.075693, -0.424958,  0.031507, -0.214998, -0.11862 , -0.059799,
       -0.150971, -0.176567,  0.444136,  0.18925 ,  0.241265, -0.248733,
       -0.176206,  0.579075,  0.031083,  0.253453,  0.101754,  0.144021,
       -0.169662, -0.005122,  0.056225,  0.078038, -0.557848, -0.365922,
       -0.108863,  0.322074, -0.178232,  0.289982, -0.205699,  0.001991,
        0.001029, -0.074944,  0.173145,  0.044346, -0.032673, -0.060124,
        0.152098, -0.293121, -0.280878,  0.377744,  0.402627, -0.737609,
        0.020663, -0.01301 ,  0.134813, -0.052369,  0.046645, -0.024955,
        0.100392, -0.295437, -0.418354,  0.396864,  0.232666,  0.022819,
       -0.018474,  0.385309, -0.084371,  0.269105,  0.149824, -0.161652,
        0.282115,  0.177849, -0.241524, -0.240634, -0.111456,  0.070668,
       -0.216499, -0.139156, -0.576622, -0.136423,  0.403575,  0.155968,
        0.218948, -0.002413,  0.123077,  0.102393,  0.741444,  0.390994,
       -0.110964,  0.134329,  0.265171,  0.251443, -0.221384, -0.344603,
        0.283788,  0.284915,  0.057016,  0.384028, -0.733822,  0.303532,
       -0.583847, -0.02494 , -0.218198,  0.018208,  0.145667,  0.255729,
        0.498478, -0.112536, -0.047852, -0.337887,  0.031235, -0.432209,
        0.208977,  0.012221,  0.19098 , -0.206233, -0.113016,  0.111511,
       -0.059505,  0.027593, -0.100111, -0.230436, -0.078137,  0.278976,
        0.19794 ,  0.210459, -0.155359, -0.112397,  0.26439 , -0.199999,
        0.058368,  0.032107,  0.110054,  0.051876, -0.240713,  0.277577,
        0.289985, -0.200094, -0.101406,  0.131032, -0.059501,  0.134268,
        0.148928, -0.085373,  0.030669,  0.068627, -0.215063, -0.081981,
       -0.052877, -0.291941, -0.458875,  0.050618,  0.214273,  0.184849,
       -0.038208,  0.036236, -0.642125, -0.464941,  0.372085, -0.187011,
       -0.069729,  0.035051,  0.099067,  0.057524,  0.006579,  0.533439,
       -0.198236, -0.344194, -0.067796, -0.063289, -0.069952, -0.041706,
        0.142563,  0.05342 , -0.054871,  0.2577  ,  0.50673 ,  0.208496,
        0.650207, -0.252643,  0.320542, -0.54734 , -0.637634, -0.51038 ,
        0.025401,  0.05803 ,  0.16656 ,  0.257388,  0.305958, -0.219199,
        0.2086  , -0.251887, -0.185647, -0.027838, -0.186104, -0.070295,
       -0.094713,  0.102284, -0.123802,  0.083343,  0.315851, -0.031123,
        0.238093,  0.149717, -0.004316, -0.181932, -0.086956,  0.531083,
        0.112422,  0.231151,  0.242227,  0.012977, -0.095725, -0.283862,
       -0.061226,  0.311026,  0.146489,  0.054716, -0.048658, -0.282848,
       -0.054836, -0.030167, -0.090273, -0.435833, -0.287074, -0.351567,
       -0.002671, -0.020709, -0.276286,  0.281762, -0.114932, -0.243566,
       -0.188988,  0.007132,  0.146958, -0.269132, -0.034678,  0.164219,
       -0.090405,  0.044055, -0.405617,  0.162068,  0.008641,  0.142725,
       -0.065804,  0.136329,  0.247081, -0.144136, -0.007592,  0.113807,
        0.14899 , -0.012305, -0.346247,  0.111213,  0.011855,  0.250259,
        0.068319,  0.100238,  0.010446,  0.127471, -0.033004, -0.079141,
       -0.531317,  0.318433,  0.562701,  0.354442,  0.001789,  0.171425,
        0.010197,  0.043681, -0.031715,  0.505568, -0.453908,  0.050882,
       -0.294158, -0.279005, -0.121981,  0.182517,  0.419662,  0.325643,
       -0.027132, -0.020383,  0.133923, -0.047128,  0.406531, -0.385169,
       -0.148565, -0.109004, -0.062878, -0.236207,  0.008543,  0.044215,
        0.216761,  0.017197, -0.034889,  0.311272,  0.130243, -0.122846],
      dtype=float32)
```

Znajdziemy najbardziej podobne słowo do słowa kobieta.

In [6]:
```
wv.most_similar('kobieta')
```

Out[6]:
```
[('mężczyzna', 0.7043285369873047),
 ('dziewczyna', 0.6813511252403259),
 ('ciężarna', 0.612939715385437),
 ('dziewczynka', 0.6094770431518555),
 ('50-letnia', 0.5805984139442444),
 ('młoda', 0.5788260102272034),
 ('40-letnia', 0.5631040930747986),
 ('kobietą', 0.5611501336097717),
 ('dama', 0.5526077747344971),
 ('mężczyzną', 0.5519219636917114)]
```

Znajdź najbardziej podobne słowo do słowa "kobieta", które nie jest podobne do słowa "mężczyzna"

In [7]:
```
wv.most_similar(positive=['kobieta'], negative=['mężczyzna'])
```

Out[7]:
```
[('Teresa', 0.3597542345523834),
 ('Barbara', 0.33941537141799927),
 ('Helena', 0.3383762836456299),
 ('Eleonora', 0.32829549908638),
 ('mianowana', 0.3252837359905243),
 ('czarnoskóra', 0.3224010467529297),
 ('Izabela', 0.3222866654396057),
 ('Maria', 0.32095712423324585),
 ('Jadwiga', 0.31885889172554016),
 ('Anna', 0.3188304901123047)]
```

W grupie "pies kot żubr budyń" znajdź słowo, które nie pasuje.

In [8]:
```
wv.doesnt_match(['pies', 'kot', 'żubr', 'budyń'])
```

Out[8]:
```
'budyń'
```

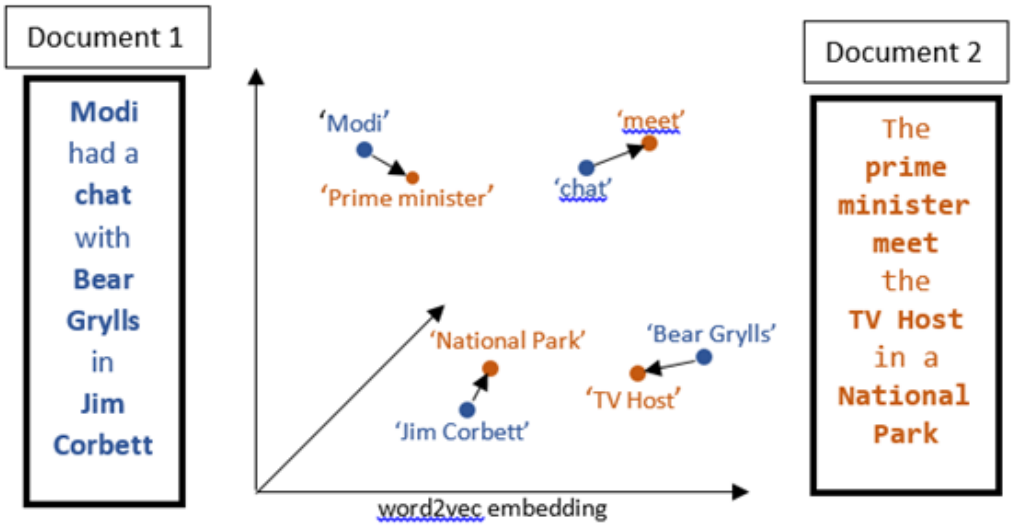
zbadaj podobieństwo między kotem a psem

In [9]:
```
wv.similarity('kot', 'pies')
```

Out[9]:
```
np.float32(0.6970217)
```

Teraz zajmiemy się podobieństwami pomiędzy zdaniami

Word mover distance to minimalna suma odległości, jaką trzeba „pokonać", aby słowa z jednego zdania pokryły znaczeniowo słowa z drugiego zdania.



In [10]:
```
!pip install POT #Po tej instalacji gensim może szybciej i dokładniej liczyć
```

```
Collecting POT
  Downloading pot-0.9.7.post1-cp313-cp313-macosx_11_0_arm64.whl.metadata (44
kB)
Requirement already satisfied: numpy>=1.16 in /Users/oskartarka/Informatyka/
Uczelnia/Aktualne przedmioty/PJN-cwiczenia/venv/lib/python3.13/site-packages
(from POT) (2.5.4)
Requirement already satisfied: scipy>=1.6 in /Users/oskartarka/Informatyka/U
czelnia/Aktualne przedmioty/PJN-cwiczenia/venv/lib/python3.13/site-packages
(from POT) (1.18.1)
Downloading pot-0.9.7.post1-cp313-cp313-macosx_11_0_arm64.whl (923 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 923.3/923.3 kB 9.4 MB/s  0:00:00
Installing collected packages: POT
Successfully installed POT-0.9.7.post1

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
```

In [11]:
```
sentence_1 = 'Byłem dzisiaj w kinie'.lower().split()
sentence_2 = 'Poszłam wczoraj do teatru'.lower().split()
```

In [12]:
```
wv.wmdistance(sentence_1, sentence_2)
```

Out[12]:
```
1.176677407878612
```

WMD nie jest ograniczone do 0-1, ale przy normalizacji embeddingów zwykle wartości mieszczą się w 0-4 dla krótkich zdań.

wynik ~1.25 sugeruje, że zdania są umiarkowanie podobne semantycznie.

Dla porównania:

- Dwa identyczne zdania → WMD ≈ 0
- Zupełnie różne zdania → WMD > 2.5-3

Flair

Flair to biblioteka NLP od Zalando Research (Python)

Główna idea:

- Klasyczne embeddingi (Word2Vec, GloVe, FastText) dają stały wektor dla każdego słowa, niezależnie od kontekstu:

  "bank" w *"river bank"* i *"money bank"* → ten sam wektor.

- Flair tworzy kontekstowe embeddingi(wektor słowa zależy od jego zdania / kontekstu)

  więc "*bank*" w "*river bank*" będzie inny niż w "*money bank*".

Flair uczy się tego przez modele LSTM działające w dwóch kierunkach (forward i backward).

In [75]:
```
#Tak samo
#!pip install flair
```

In [20]:
```
from flair.embeddings import FlairEmbeddings, StackedEmbeddings
from flair.data import Sentence
```

In [21]:
```
embeddings = FlairEmbeddings('news-forward')

#tworzymy objekty Sentence
s1 = Sentence("How does it feel to be a scribe?")
s2 = Sentence("There is no such thing as simply good or bad")
sentences = [s1, s2]


# Obliczamy embeddingi(zanurzamy słowa)
embeddings.embed(sentences)

for sentence in sentences: #Zanurzone zdania nadal są obiektem Sentence
  print(sentence)
```

```
Sentence[9]: "How does it feel to be a scribe?"
Sentence[10]: "There is no such thing as simply good or bad"
```

In [22]:
```
#dostajemy embeddingi poszczególnych słów
for sentence in sentences:
  for token in sentence:
    print(token.embedding)
#embeddingi są tensorami
```

```
tensor([-0.0011, -0.0021,  0.0038,  ..., -0.0004,  0.0099,  0.0547])
tensor([-0.0016, -0.0260,  0.0388,  ...,  0.0031, -0.0070,  0.0025])
tensor([0.0174, 0.0181, 0.0148,  ..., 0.0051, 0.0051, 0.0130])
tensor([ 0.0001,  0.0128,  0.0476,  ..., -0.0084, -0.0023,  0.0106])
tensor([-2.1672e-03,  8.3752e-05,  2.5102e-02,  ..., -2.8347e-04,
        -1.2090e-02,  2.9513e-02])
tensor([-0.0032,  0.0020,  0.0280,  ...,  0.0004, -0.0017, -0.0106])
tensor([ 5.3283e-03,  3.8049e-05, -2.2042e-02,  ..., -4.1646e-04,
         1.9279e-02, -1.2351e-02])
tensor([-1.0594e-03,  4.8669e-05,  2.1462e-02,  ..., -1.1294e-05,
        -4.3308e-03,  3.1280e-03])
tensor([ 8.7317e-04,  3.7861e-06, -4.6359e-04,  ..., -1.4222e-04,
         5.4708e-03,  3.0888e-03])
tensor([-0.0012, -0.0002, -0.1499,  ...,  0.0024,  0.0017,  0.1648])
tensor([ 0.0078, -0.0047,  0.0564,  ..., -0.0012,  0.0325,  0.0035])
tensor([ 5.8073e-02, -7.9190e-05,  9.8895e-02,  ..., -2.7518e-03,
         8.8326e-04,  2.6957e-02])
tensor([-0.0004, -0.0036,  0.0781,  ..., -0.0004, -0.0344, -0.0005])
tensor([ 0.0014,  0.0033, -0.0118,  ..., -0.0005, -0.0164,  0.0293])
tensor([ 1.2536e-02, -1.0569e-03, -1.1876e-02,  ..., -2.9878e-05,
         4.0988e-03,  2.8744e-02])
tensor([-0.0053, -0.0011,  0.0058,  ..., -0.0018,  0.0212,  0.0082])
tensor([ 2.0060e-04,  9.6346e-04, -2.2168e-02,  ...,  6.8235e-05,
        -1.3982e-01,  3.5051e-03])
tensor([-9.0033e-05, -1.7683e-03,  1.0521e-02,  ..., -1.9229e-03,
        -1.2466e-05,  3.2030e-02])
tensor([-0.0006,  0.0004, -0.0019,  ..., -0.0005,  0.0061, -0.0073])
```

In [23]:
```
import numpy as np
```

In [24]:
```
for sentence in sentences:
  for token in sentence:
    print(np.array(token.embedding))
```

```
[-0.00108573 -0.00213396  0.00377837 ... -0.00035715  0.00994328
  0.05467629]
[-0.00163258 -0.02604981  0.03880603 ...  0.00308893 -0.00699768
  0.00248321]
[0.01740975 0.01807806 0.01477519 ... 0.00507008 0.00508687 0.01302868]
[ 0.00013419  0.01282793  0.04760627 ... -0.00837355 -0.00231261
  0.01057429]
[-2.1671706e-03  8.3752457e-05  2.5102198e-02 ... -2.8347090e-04
 -1.2090319e-02  2.9513311e-02]
[-0.00319913  0.00195263  0.02804708 ...  0.00040722 -0.00174229
 -0.01062472]
[ 5.3283228e-03  3.8049242e-05 -2.2042057e-02 ... -4.1645759e-04
  1.9278506e-02 -1.2350944e-02]
[-1.0593978e-03  4.8669139e-05  2.1461954e-02 ... -1.1294099e-05
 -4.3307603e-03  3.1279747e-03]
[ 8.7317172e-04  3.7861441e-06 -4.6359180e-04 ... -1.4221967e-04
  5.4708058e-03  3.0888284e-03]
[-0.00118427 -0.00018427 -0.14988402 ...  0.00244022  0.00166775
  0.16482718]
[ 0.00780445 -0.00474085  0.05643408 ... -0.00120922  0.03250846
  0.00350803]
[ 5.8073457e-02 -7.9189660e-05  9.8894536e-02 ... -2.7517839e-03
  8.8326150e-04  2.6957313e-02]
[-0.0003961  -0.00361207  0.07808591 ... -0.00039113 -0.03444924
 -0.00053955]
[ 0.00144947  0.00326681 -0.01180755 ... -0.00050382 -0.0164138
  0.0292847 ]
[ 1.2535829e-02 -1.0568543e-03 -1.1875675e-02 ... -2.9878074e-05
  4.0988061e-03  2.8744429e-02]
[-0.00529047 -0.00108928  0.0058023  ... -0.00176484  0.02117527
  0.00820003]
[ 2.0059792e-04  9.6345594e-04 -2.2167904e-02 ...  6.8235255e-05
 -1.3981704e-01  3.5051247e-03]
[-9.0033063e-05 -1.7682664e-03  1.0520662e-02 ... -1.9228812e-03
 -1.2465802e-05  3.2029983e-02]
[-0.00058445  0.00037377 -0.00193681 ... -0.00046098  0.00614755
 -0.00726031]
/var/folders/w0/z7hwst390mq37hgy25gqy5980000gn/T/ipykernel_2314/932037997.p
y:3: DeprecationWarning: __array__ implementation doesn't accept a copy keyw
ord, so passing copy=False failed. __array__ must implement 'dtype' and 'cop
y' keyword arguments. To learn more, see the migration guide https://numpy.o
rg/devdocs/numpy_2_0_migration_guide.html#adapting-to-changes-in-the-copy-ke
yword
  print(np.array(token.embedding))
```

Embeddingi dla zdań

In [25]:
```
from flair.embeddings import DocumentPoolEmbeddings
```

In [26]:
```
sentence_embeddings = DocumentPoolEmbeddings([FlairEmbeddings('news-forward'
```

In [27]:
```
sentence_embeddings.embed(sentences)
```

In [28]:
```
for sentence in sentences:
  print(sentence.embedding)
```

```
tensor([ 0.0016,  0.0005,  0.0175,  ..., -0.0001,  0.0014,  0.0104])
tensor([ 0.0073, -0.0008,  0.0052,  ..., -0.0007, -0.0124,  0.0289])
```

Zadania

Zadanie1

Załaduj polski model kontekstowych embeddingów Flair:

In [29]:
```
embeddings = FlairEmbeddings('pl-forward')
```

```
2026-10-14 23:44:20,433 https://flair.informatik.hu-berlin.de/resources/embeddings/flair/lm-polish-forward-v0.2.pt not found in cache, downloading to /var/folders/w0/z7hwst390mq37hgy25gqy5980000gn/T/tmp35kmmnwg
100%|██████████| 80.3M/80.3M [00:02<00:00, 33.1MB/s]
2026-10-14 23:44:23,213 copying /var/folders/w0/z7hwst390mq37hgy25gqy5980000gn/T/tmp35kmmnwg to cache at /Users/oskartarka/.flair/embeddings/lm-polish-forward-v0.2.pt
2026-10-14 23:44:23,239 removing temp file /var/folders/w0/z7hwst390mq37hgy25gqy5980000gn/T/tmp35kmmnwg
```

In [30]:
```
s1 = Sentence("Zamek w Malborku jest bardzo stary.")
s2 = Sentence("Suwak w kurtce to zamek błyskawiczny")
s3 = Sentence("Sprawdź, czy zamek w drzwiach działa")
sentences = [s1, s2, s3]
```

Oblicz embeddingi dla wszystkich zdań:

In [31]:
```
embeddings.embed(sentences)
```

Out[31]:
```
[Sentence[7]: "Zamek w Malborku jest bardzo stary.",
 Sentence[6]: "Suwak w kurtce to zamek błyskawiczny",
 Sentence[7]: "Sprawdź, czy zamek w drzwiach działa"]
```

Wypisz każde zdanie i dla słowa "zamek", wypisz pierwsze 5 wymiarów embeddingu:

In [34]:
```
for sentence in sentences:
    print(sentence)
    for token in sentence:
        if token.text.lower() == "zamek":
            print(token.embedding[5])
```

```
Sentence[7]: "Zamek w Malborku jest bardzo stary."
tensor(-0.0006)
Sentence[6]: "Suwak w kurtce to zamek błyskawiczny"
tensor(-0.0005)
Sentence[7]: "Sprawdź, czy zamek w drzwiach działa"
tensor(-0.0009)
```

Czy embeddingi są różne w zależności od znaczenia słowa?

Odpowiedź: Tak

Zadanie 2

Stwórz bota z bazą przykładownych pytań i odpowiedzi, zwracającego odpowiedź na zapytanie na podstawie podobieństwa (WMD albo flair document embeddings) pomiędzy zapytniem użytownika a pytaniami w swojej bazie

Przykład:

Zbiór pytań i odpowiedzi :

```
[
  {"q1" : "a1"},
  {"q2" : "a2"},
  {"q3" : "a3"},
  ...
  {"qn" : "an"},
]
```

Wektoryzujemy pytania:

```
  {"q1_vecotorized" : "a1"},
  {"q2_vecotorized" : "a2"},
  {"q3_vecotorized" : "a3"},
  ...
  {"qn_vecotorized" : "an"},
```

Dla pytania użytkownia pytania q_user wektoryzujemy je i liczmy podobieństwo z pytaniami w bazie

```
sim("q_user_vectorized","q1_vecotorized"),
sim("q_user_vectorized","q2_vecotorized"), ...
("q_user_vectorized""qn_vecotorized")
```

Zwracamy odpowiedź do pytania z bazy najbardziej podobnego do pytania użytkownika $$ argmax(sim(q_{user}vectorized,q_{i}vecotorized)) $$

Do liczenia podobnieństma można użyć np https://docs.scipy.org/doc/scipy/reference/

generated/scipy.spatial.distance.cosine.html ale wtedy robimy argmin bo $$ cosine\ distance = 1 - cosine\ similarity $$

In [69]:
```
qa = [
  {"q": "Czym jest PC?", "a": "PC to 'Personal Computer' z języka angielskie
  {"q": "Dlaczego RAM, SSD i karty graficzne tak duzo kosztuja kosztuja?", "
  {"q": "Czy sztuczna inteligencja zastapi nas?", "a": "Istnieje bardzo duża
  {"q": "Czym jest kłamstwo?", "a": "Świadome powiedzenie nieprawdy lub wpro
  {"q": "Jak dlugo powinienem spać?", "a": "Zalecana dawka snu to osiem godz
]
```

In [70]:
```
# importy
from scipy.spatial.distance import cosine

doc_em = DocumentPoolEmbeddings([FlairEmbeddings('pl-forward')])

def calculate_similarity(vec1, vec2):
    return 1 - cosine(vec1, vec2)

def embed_text(text):
    s = Sentence(text)
    doc_em.embed(s)
    return s.embedding.detach().cpu().numpy()

qa_vectors = [embed_text(item["q"]) for item in qa]

def bot(user_question):
    user_vec = embed_text(user_question)
    similarities = [calculate_similarity(user_vec, v) for v in qa_vectors]
    best = max(range(len(qa)), key=lambda i: similarities[i])

    print(f"Pytanie użytkownika: {user_question}")
    print(f"Najbardziej podobne pytanie z bazy danych: {qa[best]['q']} (podo
    print(f"Odpowiedź: {qa[best]['a']}")
    print()


def calculate_similarity(vec1, vec2):
    return 1 - cosine(vec1, vec2)
```

In [71]:
```
bot("O której godzinie mam wstać jak idę spać o 00:00?")
bot("Jest jakiś powód dlaczego SSD i RAM sa takie drogie?")
bot("Wprowadzenie w blad to to samo co kłamstwo?")
```

```
Pytanie użytkownika: O której godzinie mam wstać jak idę spać o 00:00?
Najbardziej podobne pytanie z bazy danych: Jak dlugo powinienem spać? (podob
ieństwo: 0.743)
Odpowiedź: Zalecana dawka snu to osiem godzinn dzinnie

Pytanie użytkownika: Jest jakiś powód dlaczego SSD i RAM sa takie drogie?
Najbardziej podobne pytanie z bazy danych: Dlaczego RAM, SSD i karty graficz
ne tak duzo kosztuja kosztuja? (podobieństwo: 0.783)
Odpowiedź: Ponieważ boom na AI spowodował, że producenci skupili się na prod
ukcji dla wielkich korporacji

Pytanie użytkownika: Wprowadzenie w blad to to samo co kłamstwo?
Najbardziej podobne pytanie z bazy danych: Czym jest kłamstwo? (podobieństw
o: 0.675)
Odpowiedź: Świadome powiedzenie nieprawdy lub wprowadzenie innej osoby w bła
d
```

In [ ]: